

Simulacro ORDERS

EJERCICIO 1.

- 1 Añadimos en orderScreen el estado para los pedidos `const [orders, setOrders] = useState([])`
- 2 En el `useEffect` añadimos `fetchRestaurantOrders()`
- 3 Hacemos `fetchRestaurantOrders`
- 4 Hacemos `renderOrder`
- 5 Devolvemos la `FlatList` configurada

EJERCICIO 2.

- 1 Añadir el botón de edición en `OrdersScreen.js` (dentro de `renderOrder`)
- 2 Implementar `EditOrderScreen.js`
 - `useEstate order`
 - `initialValues`
 - `backendErrors`
 -
 - `validationSchema`
 - `useEffect` (usando `route.params.orderId` -ya que está dentro de un restaurante-)
 - `updateOrder`
 - `Formik`
 -
 - *Añadir `EditOrderScreen` a `RestaurantStack`

EJERCICIO 3

- 1 `const [analytics, setAnalytics] = useState(null)`
- 2 `fetchRestaurantAnalytics` (usando `getRestaurantAnalytics`)
 - `getRestaurantAnalytics` hay que hacerlo en `RestaurantEndpoints`
- 3 En `renderAnalytics` cambiamos los `TODO` por las propiedades del objeto que devuelve el backend

4 Añadir renderAnalytics() en renderHeader (Porque esta en la cabecera de la pantalla)

EJERCICIO 4

1 Hacemos el handlerNextStatus

2 Añadir el boton en renderOrder

3 Añadir <view style={styles.categoryRow}> en render order para que se vea los botones.

SIMULACRO SCHEDULES

1 Tenemos que tocar el renderProduct. `{item.schedule ? <> ... </> :<> ...</>}` Es como un if. Si existe item.schedule pasa algo (...) si no pasa lo otro (...).

Ponemos la imagen y el texto en verde si existe el item.schedule. Si no existe ponemos la imagen y el texto en rojo (Not schedule). Poner si un item no esta disponible (item.availability) el not Available.

2 Hacemos la función de fetchSchedules para traernos los schedules del backend.

Editamos cada schedule en el renderSchedule (startTime, endTime, contador de productos y botones)

3 Realizamos la función remove de schedule usando schedule.restaurantId y [schedule.id](#). Ponemos el deleteModal en el return después del FlatList

4 Ponemos el backendErrors (const [backendErrors, setBackendErrors] = useState()) y el const [schedule, setSchedule] = useState() y los valores iniciales (const [initialScheduleValues, setInitialScheduleValues] = useState({ startTime: null, endTime: null })))

Realizamos el validationSchema (Aqui no se porque no se pone el restaurantId)

Sólo se pone el id en validationSchema si el usuario va a introducirlo/editarlo. Por ejemplo, cuando se crea un restaurante hay que elegir el restaurantCategory, para ello hacemos el validationSchema de restaurantCategoryId (como integer, ya que para nosotros es 1, 2..., pero para el usuario es Italiano, Mediterráneo...)

Hacemos el useEffect con fetchScheduleDetail

Realizamos el update y el return

1. ¿Cuándo poner el **useEffect** (y el **fetch**)?

El **useEffect** es el "disparador automático". Sirve para decirle a la pantalla: "En cuanto aparezcas, haz esto sin que el usuario toque nada".

Tipo de Pantalla	¿Lleva <code>useEffect</code> + <code>fetch</code> ?	¿Por qué?
Listado (ej: <code>OrdersScreen</code>)	SÍ	Porque necesitas pedirle al servidor todos los elementos para pintarlos en la lista nada más entrar.
Detalle (ej: <code>RestaurantDetail</code>)	SÍ	Porque necesitas los datos frescos de ese restaurante concreto usando su ID.
Editar (ej: <code>EditOrderScreen</code>)	SÍ	Imprescindible. Necesitas saber qué hay escrito ahora mismo en la base de datos para rellenar el formulario.
Crear (ej: <code>CreateProduct</code>)	A VECES	No para el producto (porque está vacío), pero sí si necesitas cargar un desplegable de categorías, por ejemplo.

Para poner un texto:

```
<MaterialCommunityIcons name='timetable'
color={GlobalStyles.brandGreen} size={20}/>
```

*Los colores con `GlobalStyles(...)`

Si queremos poner un `if` con varios elementos debemos de seguir esta estructura:

```
{item.schedule ? <>
  <MaterialCommunityIcons name='timetable' color={GlobalStyles.brandGreen} size={20}/>
  <TextRegular textStyle={{color: GlobalStyles.brandGreen}}>{item.schedule.startTime}-{item.schedule.endTime}</TextRegular>
</>
: <>
  <MaterialCommunityIcons name='timetable' color={GlobalStyles.brandPrimary} size={20}/>
  <TextRegular textStyle={{color: GlobalStyles.brandPrimary}}>{item.schedule.startTime}-{item.schedule.endTime}</TextRegular>
</>
}
```

Views importantes para dar formato:

- Cuando queremos que aparezca una imagen de MaterialCommunityIcons seguido de un texto:

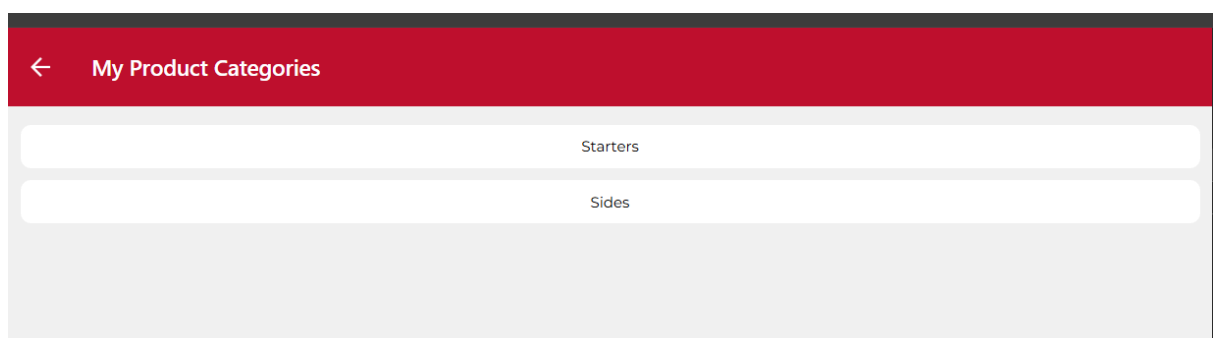
```
<TextSemiBold textStyle={styles.price}>{item.price.toFixed(2)}</TextSemiBold>  
<View style={{ flexDirection: 'row', justifyContent: 'space-between' }}>  
  <View style={{ flexDirection: 'row', alignItems: 'center' }}>
```

- Cuando queremos poner dos elementos (StartTime: 19:00:00). Row: para ponerlo uno al lado de otro no en columna y flex para que ocupe todo el horizontal. Con *justifyContent: 'flex-end'* lo ponemos en el lado derecho de la pantalla

```
<View>  
  <View style={{ flex: 1, flexDirection: 'row' }}>  
    <TextSemiBold>Start Time:</TextSemiBold>  
    <TextRegular textStyle={{ marginLeft: 5, color: GlobalStyles.brandGreen }}>{item.startTime}</TextRegular>  
  </View>  
  <View style={{ flex: 1, flexDirection: 'row' }}>  
    <TextSemiBold>End Time:</TextSemiBold>  
    <TextRegular textStyle={{ marginLeft: 5, color: GlobalStyles.brandPrimary }}>{item.endTime}</TextRegular>  
  </View>  
</View>
```

En styles también estará el fondo donde van los nombres (pedidos, productos...). Lo hacemos con un view y un style={styles.(...)}. Lo

```
const renderCategory = ({ item }) => {  
  return (  
    <View style={styles.categoryRow}>  
      <TextRegular textStyle={styles.categoryName}>{item.name}</TextRegular>  
    </View>  
  )  
}
```



LISTA DE ESTILOS:

1. El Layout (Estructura y Posicionamiento)

Estas son las más importantes para que los elementos no aparezcan amontonados.

- **flex: 1**: Le dice al componente: "ocupa todo el espacio libre que puedas". Se usa en el contenedor principal para que la pantalla no se quede "cortada" a mitad.
 - **flexDirection: 'row'**: Por defecto, React Native pone todo en columna (uno debajo de otro). Con esto, los pones en **fila** (uno al lado del otro).
 - **justifyContent**: Alinea en el eje principal.
 - **'center'**: Todo al medio.
 - **'space-between'**: Empuja los elementos a los extremos (muy usado para poner el Título a la izquierda y el Precio a la derecha).
 - **alignItems: 'center'**: Alinea los elementos en el eje contrario. Si estás en una fila (**row**), esto hace que el texto y los iconos queden **centrados verticalmente** entre sí.
 - **gap: 10**: Crea una separación automática de 10 píxeles entre cada hijo del contenedor. Es mucho más limpio que poner márgenes individuales.
-

2. Espaciado (Padding vs Margin)

- **padding: 20**: Espacio **dentro** del componente (entre el borde y el contenido).
 - **paddingVertical** / **paddingHorizontal**: Atajos. El vertical afecta a arriba/abajo; el horizontal a izquierda/derecha.
 - **marginBottom: 10**: Espacio **fuera** del componente, específicamente hacia abajo, para separar este elemento del que viene después.
 - **marginTop: 15**: Espacio fuera hacia arriba.
-

3. Estética y Bordes

- **borderRadius: 8**: Suaviza las esquinas. Cuanto más alto el número, más redondeado (un botón con 8-10 queda muy profesional).
 - **borderBottomWidth: 1**: Dibuja una línea solo en la parte de abajo. Se usa para separar elementos en una lista (como las direcciones o pedidos).
 - **borderColor: '#ddd'**: El color de esa línea o borde.
-

4. Estilos de Texto

- **fontSize: 20**: El tamaño de la letra.

- **textAlign: 'center'**: Centra el texto dentro de su propio bloque (muy importante para los textos de los botones o mensajes de "Lista vacía").
- **color: brandSecondary**: Cambia el color del texto. En el examen, usa siempre las variables de **GlobalStyles** para que el profesor vea que sigues el libro de estil

<https://github.com/orgs/IISI2-IS-2025/repositories>